Improving ICU Patient Bed Moves Policies with Reinforcement Learning Based Simulation-Optimisation

Geraint I. Palmer-Liyu Mark Tuson Virginia Ahedo Paul R. Harper
Mathew Pugh James Williams Matt Morgan

August 25, 2026

1 Introduction

2 Background

TODO

Here we need some background on the ICU department at the Heath; background and literature on on ICU bed moves, why we move patients, why it's bad to move patients, etc.

3 ICU Description

The ICU at UHW consists of 34 beds across 4 wards (named A3 North, A3 South, B3 North, and B3 South), two single isolation units, and one double isolation unit. Two beds are generally ring-fenced, left unoccupied, for emergencies. In each ward there is a partial wall partitioning the beds into two halves. This is shown in Figure 1. There are two additional units, Complex Care and the Post Anaesthetic Care Unit (PACU), which generally accommodate their own patient types (complex care and post anaesthetic patients respectively), spare beds can be used as surge, or overflow, for general ICU patients when needed.

The model will consider three types of patient, categorised as Stage 2, Stage 3, or Stage 3-I patients:

- Stage 2: these are the least severe patients. When two Stage 2 patients occupy neighbouring beds, both can be cared for by the same FTE nurse.
- Stage 3: these patients require one to one attention from nurses, regardless of patients in neighbouring beds;
- Stage 3-I: these patients require isolation and one to one attention. When an isolation unit is available, they must be admitted there. If an isolation unit is occupied by a Stage 2 or Stage 3 patient, and non-isolation bed is available, then the other patient must be moved to the available bed and the new Stage 3-I patient must be admitted to the isolation unit. If they cannot be placed in an isolation unit at all, they are placed in a shared unit, and will require their own FTE nurse similar to Stage 3 patients. They will also incur a penalty cost for each time unit they are placed in a shared block.

In order to make the problem tractable, we make the following simplifications and assumptions in the model:

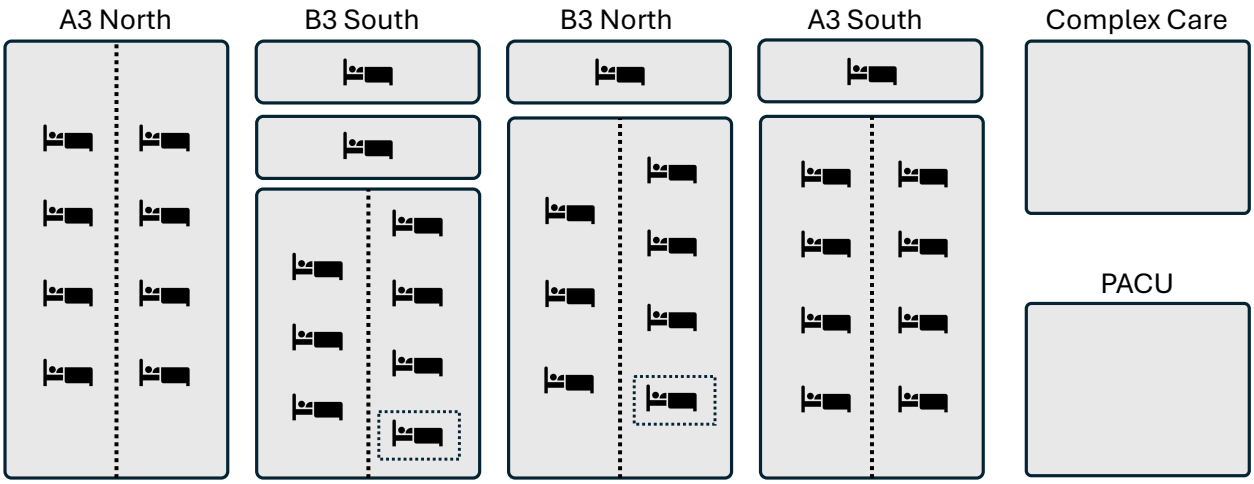


Figure 1: Representation of the four ICU wards. Solid blocks represent wards and isolation units, dotted lines represent partial walls within a ward, and the dotted boxes represent the ring-fenced beds.

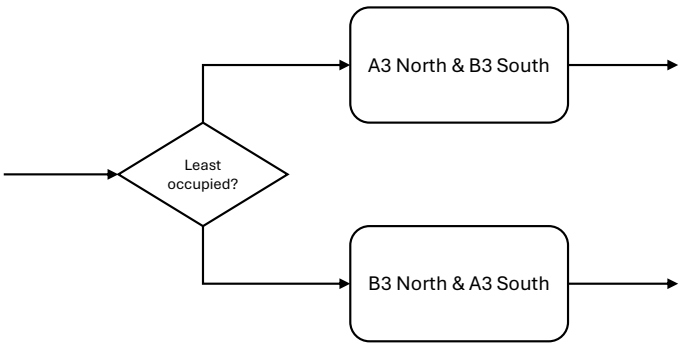
- the double isolation unit is treated as two separate isolation units. The only time this assumption may be violated would be if occupied by a patient with a contagious disease, in which case the second bed can then only be occupied by a patient with the same contagious disease; This is an uncommon enough occurrence to ignore for modelling purposes.
- general ICU patients are not admitted to Complex Care or PACU, if there are beds available in the other four wards. However, Stage 2 patients may be moved there as surge. This is generally undesirable, and so incurs some penalty;
- a nurse cannot care for two adjacent Stage 2 patients if there is a partial wall between the beds;
- the ring-fenced beds are never used, they will not be modelled.

Data analysis shows that:

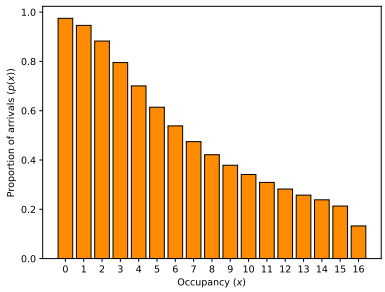
TODO

Parametrise these properly, referring back to Section 2. We can do some data analysis here.

- Stage 2 patients arrive according to the Poisson process with rate $\lambda_0 = 3.0$,
- Stage 3 patients arrive according to the Poisson process with rate $\lambda_1 = 2.0$,
- Stage 3-I patients arrive according to the Poisson process with rate $\lambda_2 = 1.0$,
- Stage 2 patients' lengths of stay are Exponentially distributed with rate $\mu_0 = 0.3$,
- Stage 3 patients' lengths of stay are Exponentially distributed with rate $\mu_1 = 0.7$,
- Stage 3-I patients' lengths of stay are Exponentially distributed with rate $\mu_2 = 0.4$,
- Stage 2 patients deteriorate into Stage 3 patients with rate δ_{01},
- Stage 3 patients deteriorate into Stage 3-I patients with rate δ_{12},
- Stage 3 patients improve into Stage 2 patients with rate δ_{10},
- Stage 3-I patients improve into Stage 3 patients with rate δ_{21},



(a) Representation of the load-balancing arrival discipline for the two identical sub-problems.



(b) The proportion of the arrivals entering a single component, given the component's occupancy.

Figure 2: Problem decomposition analysis

- Each time a patient is moved, their length of stay is extended by some fraction $\mathcal{E}$ of the mean of their length of stay distribution, and $\mathcal{E} = 0.05$.

4 Problem Decomposition

The 34 bed ICU problem can be decomposed into two sub-problems by introducing another assumption. Ignoring the ring-fenced beds, we can groupi wards A3 North and B3 South, and wards B3 North and A3 South, into two identical components each containing 16 beds. We then introduce the assumption that newly arriving patients are sent to the component with the most free beds. This corresponds to a *load-balancing* arrival discipline, a join-shortest-queue discipline that takes into consideration the number of occupied servers as well as queue length [7, 6], which, due to Poisson thinning, can be modelled using single queue analysis (SQA) with a state-dependent arrival rate. This is shown in Figure 2a.

The state-dependent arrival rate can be found from a preliminary simulation, built using Ciw [9], and parametrised as above, and observing the proportion of arrivals that enter each component at each state. The resulting proportions are given in Figure 2b. SQA now gives us that the arrival rate for each component, when the component's occupancy is x, is $\lambda_2 p(x)$, $\lambda_3 p(x)$, $\lambda_{3I} p(x)$.

We can now analyse each decomposed sub-problem separately, however each of our sub-problems are identical, so only one needs to be considered. The rest of the paper will now consider just one of the decomposed sub-problems, consisting of an ICU with two wards of 6 and 8 beds, and two isolation units, giving 16 beds in total. The 8 bed ward is partitioned in two by the partial wall, while the 6 bed wall is partitioned into two groups of 3 beds by its partial wall.

5 MDP Formulation

We will consider the bed move policy problem as a Markov Decision Process (MDP), which is a tuple (S, A, P, R), with states $s \in S$, actions $a \in A$, transition probabilities $p_{s,s',a} \in P$, and rewards $r_{s,s',a} \in R$. This is a discrete time MDP, with actions being taken at the discrete events when a new patient is admitted. The approach taken to find the optimal policy will be reinforcement learning, a model-free approach, and so the probabilities P need not be defined, and transitions are taken care of with a simulation model. The following sections describe each component.

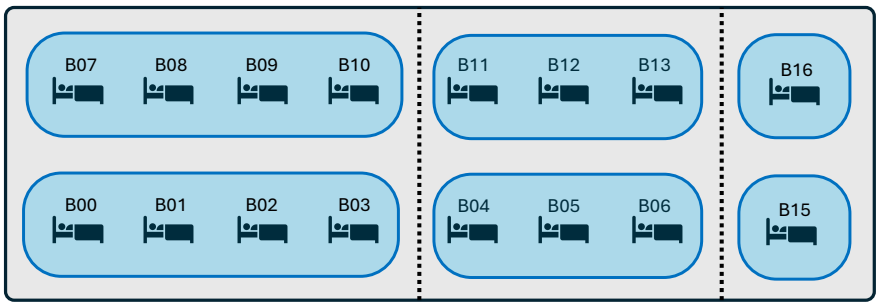


Figure 3: Representation of the 16 ICU beds partitioned into 4 blocks.

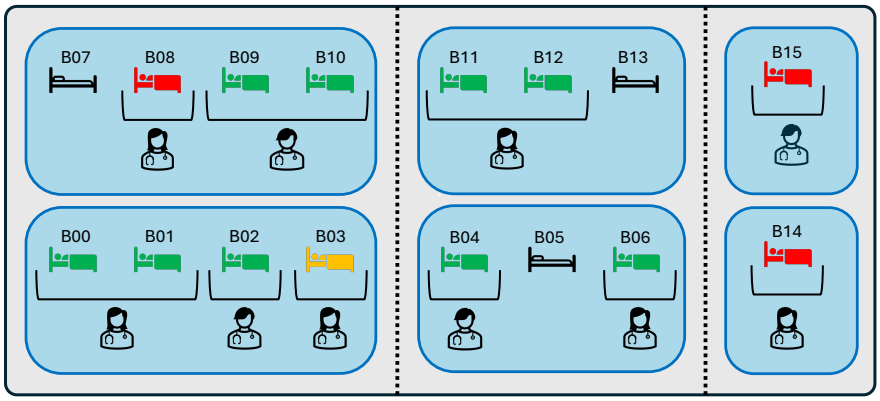


Figure 4: Representation of how nurses can share adjacent Stage 2 patients.

5.1 States

The decomposed ICU ward contains 16 beds, which can be partitioned into 4 blocks, shown in Figure 3. These blocks are defined by physical walls: beds B00 to B06 are in one room, beds B07 to B13 are in another room, and beds B14 and B15 are in isolation units. Partial walls do not count as ward separations. These blocks will determine the cost of a bed-move: intra-block moves are easier than inter-block moves, and their cost or penalty will reflect this.

Within a block the location of a patient matters: if two Stage 2 patients are in adjacent beds, it is possible for one FTE nurse to care for both. This is shown in Figure 4. Therefore the states need to hold information on both the patient type, and which particular bed within a block the patient is in. However, both isolation units are indistinguishable from one another, so there is no need to differentiate these.

Define the ward state by a 3×15 matrix M, where rows correspond to patient types (where $j = 0$ corresponds to Stage 2 patients, $j = 1$ to Stage 3 patients, and $j = 2$ to Stage 3-I patients), and columns corresponding to bed numbers (except for column 14, which corresponds to both isolation units). Then its integer entries M_{ij} denote the number of category j patients currently occupying a bed i. Note that for columns 0 to 13 the columns sums should be no more than 1, as they correspond to single beds, while for column 14 the sum should be no more than 2, as it corresponds to both isolation units. Let $\mathcal{M}$ denote the set of all possible matrices M.

As an example, consider the ICU state given in Figure 4, where green, amber and red beds indicate beds occupied by Stage 2, Stage 3 and Stage 3-I patients respectively, and black beds are unoccupied. This corresponds to the ward state:

$$M^\star = \left(\begin{array}{cccc:ccc:cccc:ccc:c} 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 2 \end{array}\right) \tag{1}$$

Ward state changes are caused by newly arriving patients of different categories being placed into beds in specific blocks, patient discharges, patient deterioration or improvement (a patient changing type), and moving a patient from one bed to another.

Actions only take place when a new patient arrives to the ward, and this patient can be either Stage 2, Stage 3, or Stage 3-I (category 0, 1, or 2). Therefore the states of the MDP also need to contain information about the newly arriving patient. Note that Stage 2 patients cannot arrive when the ward is full, while Stage 3 and Stage 3-I patients can arrive when the ward is full, only if there is a Stage 2 patient present for them to displace. Let

$$s = (M, p)$$

represent the state of the MDP, where M is a 3×15 matrix representing the ward state, and $p \in \{0, 1, 2\}$ indicate the category of the arriving patient. Note that we only need states where a patient can arrive.

Proposition 1. *Let S be the set of all possible states $s = (M, p)$. Then $|S| = 8,024,267,562$.*

Proof. First consider $|\mathcal{M}|$, the number of matrices M that exist.

- For each of the first 14 columns representing beds not in an isolation unit, $M_{i,j} \in \{0, 1\}$, and the column sum $M_{\cdot,j} \leq 1$. There are four ways to satisfy this:

  $$\left\{\begin{pmatrix}0\\0\\0\end{pmatrix}, \begin{pmatrix}1\\0\\0\end{pmatrix}, \begin{pmatrix}0\\1\\0\end{pmatrix}, \begin{pmatrix}0\\0\\1\end{pmatrix}\right\}$$

  Each column is independent of the rest, and so there are 4^{14} configurations for these columns.

- For the final column, representing the isolation units, $M_{i,15} \in \{0, 1, 2\}$, and the column sum $M_{\cdot,14} \leq 2$. There are ten ways to satisfy this:

  $$\left\{\begin{pmatrix}0\\0\\0\end{pmatrix}, \begin{pmatrix}1\\0\\0\end{pmatrix}, \begin{pmatrix}1\\1\\0\end{pmatrix}, \begin{pmatrix}1\\0\\1\end{pmatrix}, \begin{pmatrix}0\\1\\0\end{pmatrix}, \begin{pmatrix}0\\1\\1\end{pmatrix}, \begin{pmatrix}0\\0\\1\end{pmatrix}, \begin{pmatrix}2\\0\\0\end{pmatrix}, \begin{pmatrix}0\\2\\0\end{pmatrix}, \begin{pmatrix}0\\0\\2\end{pmatrix}\right\}$$

  This column is independent of the others, and so there are 10×4^{14} possible matrices M representing valid system states.

However, not all states admit an arriving patient:

- Stage 2 patients cannot arrive if the ward is full. There are 3 ways in which each of the first 14 columns can be full, and 6 ways the final column can be full, and so there are 6×3^{14} that can be discarded for when $p = 0$.
- Stage 3 and 3-I patients can arrive if the system is full, as long as there is a Stage 2 patient present to displace and send to surge. There are 2 ways in which each of the first 14 columns can be full without a Stage 2 patient, and 3 ways for the final column, therefore there are 3×2^{14} states that can be discarded when $p \in \{1, 2\}$.

Therefore the total number of states $s = (M, p)$ is:

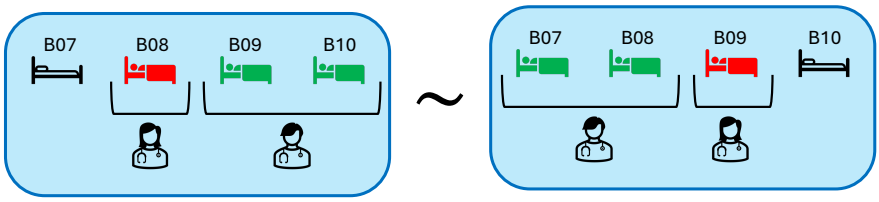


Figure 5: Example of an equivalence in a block of 4 beds.

$$\begin{aligned}|S| &= (3 \times 10 \times 4^{14}) - (2 \times 3 \times 2^{14}) - (6 \times 3^{14}) \\ &= 8,024,267,562\end{aligned}$$

□

5.1.1 State Equivalences

The state matrices exhibit some equivalences. In the two blocks of 4, and the two blocks of 3, reversed configurations are equivalent up to a bed labelling. This is shown in Figure 5. As each block of 4 beds are identical, then swapping the configurations results in equivalent configurations up to a labelling. Similarly, each block of 3 beds are identical, and so swapping their configurations result in equivalent configurations too.

Therefore define six column-transformation operations on $\mathcal{M}$ corresponding to these equivalences:

- T_1: reversing columns 0 to 3;
- T_2: reversing columns 4 to 6;
- T_3: reversing columns 7 to 10;
- T_4: reversing columns 11 to 13;
- T_5: swapping the block of columns 0–3 with the block 7–10;
- T_6: swapping the block of columns 4–6 with the block 11–13.

Let $\mathcal{T} = \{T_1, T_2, T_3, T_4, T_5, T_6\}$. We will consider all combinations of compositions of these transforms; so also consider $\langle\mathcal{T}\rangle$, the set generated by $\mathcal{T}$ and composition $\circ$, where $\langle\mathcal{T}\rangle$, is a proper subset of all possible permutations of the state's coluumns. It will be useful later to understand the inverses of the elements of $\langle\mathcal{T}\rangle$. Note that all six of these transforms are involutions, that is they are their own inverse, and applying the same transform twice will give the original matrix. As the first four concern non-overlapping columns, then they are commutative under composition, and so any combination of these, P is also an involution [3, p. 22], and $P^{-1} = P$. Similarly for compositions involving T_5 and T_6. This is not true however composing T_5 and T_1 or T_3; or composing T_6 and T_2 or T_4 as they affect the same columns. However in these cases, if T_5 (or equivalently T_6, or compositions of T_5 and T_6) were applied first or last, that is $P = \tilde{T} \circ T_5$ or $Q = T_5 \circ \tilde{T}$ where $\tilde{T}$ is some combination of the other transforms, then we can use that permutation as the inverse if we apply and re-apply T_5 (or T_6, or their compositions, respectively):

$$T_5 \circ P \circ T_5 = T_5 \circ (\tilde{T} \circ T_5) \circ T_5 = T_5 \circ \tilde{T} \circ I = T_5 \circ \tilde{T} = T_5^{-1} \circ \tilde{T}^{-1} = P^{-1}, \text{ or} \tag{2}$$

$$T_5 \circ Q \circ T_5 = T_5 \circ (T_5 \circ \tilde{T}) \circ T_5 = I \circ \tilde{T} \circ T_5 = \tilde{T} \circ T_5 = \tilde{T}^{-1} \circ T_5^{-1} = Q^{-1} \tag{3}$$

which are the exact inverse of P and Q [3, p. 18].

Now define a relation $\sim$ on $\mathcal{M}$ such that $M_1 \sim M_2$ if M_2 is obtained by applying some sequence of the column-transformations above on M_1. It is clear that $\sim$ satisfies reflexivity, symmetry, and transitivity, and is therefore an equivalence relation, and so $\mathcal{M}$ can be partitioned into disjoint equivalence classes $[M_1] = \{M_2 \in \mathcal{M} \mid M_2 \sim M_1\}$. Now define $\mathcal{M}^\star \subseteq \mathcal{M}$ to be the set containing exactly one representative matrix from each equivalence class. How this representative matrix is chosen is detailed in Section 6.1.1. Hence, we can also define $S^\star$ as the set of all possible states $s = (M, p)$ where $M \in \mathcal{M}^\star$, that is the set of all representative states.

Proposition 2. *A list of all elements of an equivalence class $[M]$ can be generated by applying all combinations of the transformations in $\mathcal{T} = \{T_1, T_2, T_3, T_4, T_5, T_6\}$ to M, where the transforms in each composition are ordered by strictly increasing indices.*

This means, for example, that $T_1 \circ T_3 \circ T_4$ is evaluated, while an alternative ordering like $T_3 \circ T_1 \circ T_2$ is excluded. $T_1 \circ T_4 \circ T_1 \circ T_2$ is also excluded as we do not repeat transformations.

Proof. By definition $[M]$ contains all elements that can be reached from M by applying any combination of transforms in $\mathcal{T}$, including repeats, in any order. That is, $[M] = \{P(M) \mid P \in \langle\mathcal{T}\rangle\}$.

- First consider a permutation P that does not use T_5 or T_6, the block-swap transformations. As all other transforms commute, we can apply the transforms in any order without changing P, and so we can obtain P by applying the transforms by strictly increasing indices. As for repeated application of any of the transforms, as they are involutions, any two consecutive repeated applications of the same transform cancel out, so P needs at most one application of each transform. Therefore P will be found by the generation scheme.
- Now consider a permutation Q that does contain at least one application of T_5 or T_6.

  First consider T_5: the only transforms that do not commute with T_5 are T_1 and T_3 as they operate on the same columns. Notice that:

$$T_5 \circ T_1 = T_3 \circ T_5$$
$$T_5 \circ T_3 = T_1 \circ T_5$$

  Similarly we see the same relationship with T_6, T_2 and T_4:

$$T_6 \circ T_2 = T_4 \circ T_6$$
$$T_6 \circ T_4 = T_2 \circ T_6$$

  From this it is always possible to re-write Q as a composition of transforms where all T_5 or T_6 transforms are always applied either first or last; and the sequence of applications of T_5 and T_6 can be reduced to either one or no applications of T_5, T_6, or $T_5 \circ T_6$, as these are involutions.

  The remainder of the sequence of transforms can be treated as P, which can be written as a sequence of unique transforms by strictly increasing indices. Therefore Q will be found by the generation scheme.

□

As there are six transformations yielding equivalent, $|[M]| \leq 2^6 = 64$. By generating all 64 permutation using Proposition 2 we ensure all equivalent states are found, though some may duplicate.

TODO

Enumerate combinatorially how many valid representative MDP states exist. Due to the five equivalences, I expect this to be around $2^6 = 64$ times smaller, not including symmetric states.

5.2 Actions

Define an action as an operator on an MDP state $s \in S$ to a ward state $M \in \mathcal{M}$. An action is placing a newly arriving patient into a bed, either unoccupied or occupied. If placing into an occupied bed, then the patient currently occupying that bed needs to be moved either into an unoccupied bed or into surge. We can denote an action by a pair of integers (b, d). When $b = d$ this denotes placing the newly arriving patient into bed b where that bed is unoccupied. When $b \neq d$ this denotes placing the newly arriving patient into bed b, and moving patient occupying bed b to bed d. When $d = 16$ this denotes moving the patient to surge.

For example, action $a = (3, 5)$ represents placing a patient into bed 3, and moving the patient currently in bed 3 to bed 6, e.g.:

$$a(M^\star, 2) = \left(\begin{array}{cccc:cccc:cccc:cccc} 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 2 \end{array}\right) \tag{4}$$

where $M^\star$ is the ward state defined in Equation 1.

Actions are state-dependent:

- only Stage 2 patients can be moved into surge;
- arriving Stage 3-I patients *must* be placed into an isolation unit if free, or they *must* move Stage 2 or Stage 3 patients out of an isolation unit if no isolation unit is free. When there is a choice of patient type to move out of an isolation unit, Stage 2 patients are moved.

The set of actions available when in a particular state $s = (M, p)$, denoted by A_s, will depend on the ward state component, M and the arriving patient type p, such that the constraints above hold. That is:

$$A_s = \{(b, d) \mid b \in \{0, \ldots, 14\},\ d \in \{0, \ldots, 15\}\} \tag{5}$$

Subject to:

$$\begin{cases} b = d \implies M_{\cdot,b} = 0 & (6) \\ b \neq d \text{ and } d < 15 \implies M_{\cdot,d} < K_d \text{ and } M_{\cdot,b} > 0 \text{ and } M_{p,b} = 0 & (7) \\ d = 15 \implies p \neq 0 \text{ and } M_{0,b} > 0 & (8) \\ p = 2 \text{ and } M_{\cdot,14} < 2 \implies b = d = 14 & (9) \\ p = 2 \text{ and } M_{\cdot,14} = 2 \text{ and } M_{2,14} \neq 2 \implies b = 14 \text{ and } (M_{\cdot,d} = 0 \text{ or } d = 15) & (10) \\ p = 2 \text{ and } M_{\cdot,14} = 2 \text{ and } M_{2,14} = 2 \implies b \neq 14 \text{ and } (M_{\cdot,d} = 0 \text{ or } d = 15) & (11) \end{cases}$$

where $K_j = 1$ for all $j < 14$, and $K_{14} = 2$. Constraint 6 corresponds to placing a newly arriving patient in an empty bed without any bed moves; constraint 7 corresponds to placing the newly arriving patient into bed b, moving the current occupier to bed d; constraint 8 corresponds to only being able to move a Stage 2 patient to surge; constraint 9 corresponds to needing to place a newly arriving Stage 3-I patlient into a free isolation unit; and constraint 10 corresponds to needing to move a less severe patient from an isolation unit to make room for an arriving Stage 3-I patient, and constraint 11 allows placing a Stage 3-I patient in

another bed only if the isolation units are full with other Stage 3-I patients. Note that constraints 7 and 8 also restrict displacing a patients of the same type, as this would result in an equivalent state to placing them directly in an empty bed.

TODO

Enumerate combinatorially how many valid MDP states-action pairs exist.

These action sets are defined for all states $s \in S$, however due to state equivalences, we only need to define them for all $s \in S^\star$. When in state $s \neq S^\star$, we transform s by permutation P into a representative of its equivalence class, such that $P(s) \in S^\star$, These state equivalences mean that when the simulation visits a particular state-action pair, the reinforcement learning agent will update the Q-value of a representative state-action pair. This is given in Proposition 3, where applying the permutation to a state corresponds to applying the permutation to the corresponding ward state matrix: $P(s) = (P(M), p)$.

Proposition 3. *For the state-action pair (s, a), the representative state-action pair is $(P(s), P^{-1}(a))$, where P is the permutation that takes the state to its representative state.*

Proof. Let s, a and P be given, such that P is the permutation that takes state s to its representative state. Let (s', a') be the representative state-action pair.

It is clear that the state component of the representative state-action pair, $s' = P(s)$, be definition of P.

If applying $a(s) = \tilde{s}$, we wish to find an action a' such that $a'(s') = \tilde{s}'$, and that $\tilde{s}'$ can be reached from $\tilde{s}$ by the same permutation P. That is:

$$P(\tilde{s}) = \tilde{s}' \tag{12}$$
$$P(a(s)) = a'(s') \tag{13}$$
$$P(a(s)) = a'(P(s)) \tag{14}$$

We wish that this relationship be true for any state s, and so we want the operations $P \circ a$ and $a' \circ P$ are equal:

$$a' \circ P = P \circ a \tag{15}$$
$$a' \circ P \circ P^{-1} = P \circ a \circ P^{-1} \tag{16}$$
$$a' \circ \mathbb{I} = P \circ a \circ P^{-1} \tag{17}$$
$$a' = P \circ a \circ P^{-1} \tag{18}$$

where Equation 18 is a conjugation, which on permutations acts as a relabelling, we have that $a' = P^{-1}(a)$ [3, p. 125], hence $(s', a') = (P(s), P^{-1}(a))$ as required. □

5.2.1 Action Equivalences

If two actions transform a state into two different yet equivalent matrices, we only need to consider one of these actions. For example consider actions $a_1 = (7, 11)$, $a_2 = (7, 13)$, $a_3 = (10, 11)$, and $a_4 = (10, 13)$ operating on a state $s = (M, 2)$, where Figure 6 shows beds B07 to B13 of M, before and after applying these actions - the resulting states are equivalent due to transforms T_3, T_4 and $T_3 \circ T_4$.

Therefore define a relation $\equiv$ such that $a_1 \equiv a_2$ if $a_2(s) \sim a_1(s)$, that is if the ward states they produce belong to the same equivalence class. It is clear that $\equiv$ also satisfies reflexivity, symmetry, and transitivity,

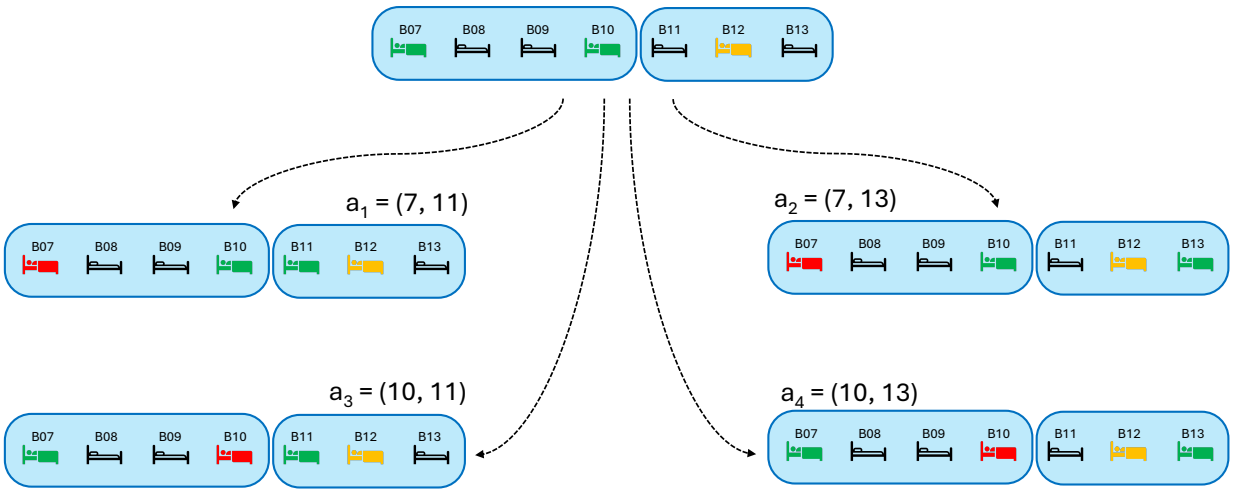


Figure 6: Example of an action equivalence.

and so is also an equivalence relation. Hence each set A_s can be partitioned into disjoint equivalence classes $[a_1] = \{a_2 \in A_s \mid a_2 \equiv a_1\}$, and so we can define $A_s^\star \subseteq A_s$ to be the set containing exactly one representative action from each equivalence class. How this representative action is chosen is detailed in Section 6.1.1.

Proposition 4. *For a state $s = (M, p)$, A_s can contain two equivalent actions when acting on s if and only if M is a fixed-point of some permutation P, that is $P(M) = M$. Under this condition, any action $a = (b, d)$ where either b or d is affected by P has an equivalent action $a' = P^{-1}(a) = (P^{-1}(b), P^{-1}(d))$.*

As a preliminary to the proof, consider that applying an action to a state is equivalent to adding to the original matrix some sparse difference matrix with elements in $\{-1, 0, 1\}$; consider the example matrix $M^\star$ with $a = (3, 5)$, we have $a(M^\star, 2) = M^\star + \delta_a$, where the matrix δ_a corresponds to removing a Stage 2 patient from bed B03, inserting a Stage 2 patient into bed B05, and inserting a Stage 3-I patient into bed B03:

$$\delta_a = \left(\begin{array}{cccc:ccc:cccc:cccc} 0 & 0 & 0 & -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}\right) \tag{19}$$

Proof. Two actions, a and a' are equivalent if, for some permutation P:

$$a(M) = P(a'(M)) \tag{20}$$
$$M + \delta_a = P(M + \delta_{a'}) \tag{21}$$
$$M + \delta_a = P(M) + P(\delta_{a'}) \tag{22}$$
$$M - P(M) = P(\delta_{a'}) - \delta_a \tag{23}$$

where Equation 22 is possible due to column permutations being linear transforms. For this to be true:

- either $P(M) = M$, and so M is a fixed-point of P, and $P(\delta_{a'}) = \delta_a$, implying that $P(a') = a$, and so $a' = P^{-1}(a)$;
- or $P(M) \neq M$. In this case the left hand side of Equation 23 must have the property that permuted columns have opposite signs. For the same property to occur in the right hand side, we must have $a' = P(a)$, but this would imply that both sides of the equation are equal to zero, hence $P(M) = M$, which is a contradiction.

Hence equivalent actions can only arise under permutations P for which M is a fixed-point, and $a = (b, d) \equiv a' = (P^{-1}(b), P^{-1}(d))$. In order for $a \neq a'$, then either b or d must be columns affected by P. □

Note that if M is a fixed-point of multiple permutations, $P_1, P_2, \ldots P_n$, then $a, P_1^{-1}(a), P_2^{-1}(a), \ldots, P_n^{-1}(a)$ are all equivalent, though not necessarily distinct.

During the simulation run, when a state is encountered, actions are generated to satisfy Equations 6- 11. In the case of equivalent actions, they are filtered out, and only representative actions are kept. In order to do this, an efficient way is needed to check which of the 64 permutations a state is a fixed-point of. We propose to do this with a maximum of eight checks.

First, define $\mathcal{H} = \{T_1, T_3, T_5, T_1 \circ T_3 \circ T_5, T_2, T_4, T_6, T_2 \circ T_4 \circ T_6\}$. This is an extension of the set $\mathcal{T}$, and is also a generator set for $\langle \mathcal{T} \rangle$. However the set $\mathcal{H}$ exhibits compositional fixed-point decomposition, which $\mathcal{T}$ did not.

Proposition 5. *For all $P_1, P_2 \in \mathcal{H}$, $P_1 \neq P_2$, we have: $(P_1 \circ P_2)(M) = M$ if and only if $P_1(M) = M$ and $P_2(M) = M$.*

Proof. This proof will first prove sufficiency, and then the necessity:

- Sufficiency: Let $P_1(M) = M$ and $P_2(M) = M$. Then $(P_1 \circ P_2)(M) = P_1(P_2(M)) = P_1(M) = M$.
- Necessity: Now let $(P_1 \circ P_2)(M) = M$. The consideration will be different depending on which columns P_1 and P_2 operate on. If P_1 and P_2 operate on different sets of columns, then this is trivially true.

  If P_1 and P_2 operate on overlapping sets of columns if they involve the block-swap transforms T_5, T_6, and their extensions $T_1 \circ T_3 \circ T_5$ and $T_2 \circ T_4 \circ T_6$. Without loss of generality consider odd-indexed transforms, which operate on the blocks of 4 beds. There are two cases:

  - If $P_1, P_2 = T_1, T_5$, then we have $(T_1 \circ T_5)(M) = M$. Therefore the second block of 4 beds is equal to the the first block of 4 beds, and the first block of 4 beds is equal to the reverse of the second block. Hence $T_5(M) = T_1(M) = T_3(M) = M$. Similarly for $P_1, P_2 = T_3, T_5$, and their reverse orderings ($P_1, P_2 = T_5, T_1$ and $P_1, P_2 = T_5, T_3$).
  - If either P_1 or P_2 is $T_1 \circ T_3 \circ T_5$, and the other is either T_1, T_3, or T_5, then the transforms, being involutions, cancel to get either $T_1 \circ T_5$, $T_3 \circ T_5$, or $T_1 \circ T_3$, all of which are covered by the cases above.

  The same argument applies for T_2, T_4, T_6, and $T_2 \circ T_4 \circ T_6$, which have the same relationship.

□

Note that if $P_1 = P_2$, then $P_1 \circ P_2$ is the identity transformation, and so $(P_1 \circ P_2)(M) = M$ even if $P_1(M) \neq M$ or $P_2(M) \neq M$,

Now in order to determine which of the permutations in $\langle \mathcal{T} \rangle$ a state M is a fixed-point of, it is sufficient to check which of the 8 elements in $\mathcal{H}$ that M is a fixed-point of. Rather than check all 8 each time, we can follow two decision trees (one for the blocks of 4 beds, and another for the blocks of 3 beds), which will reduce the number of checks from exactly 8 to between 4 and 8. The decision tree for the blocks of 4 beds in given in Figure 7, that is concerning transforms T_1, T_3, T_5, and $T_1 \circ T_3 \circ T_5$. An identical decision tree exists for the blocks of 3 beds (increasing the index of each transform by 1).

TODO

Enumerate combinatorially how many valid representative MDP states-action pairs exist.

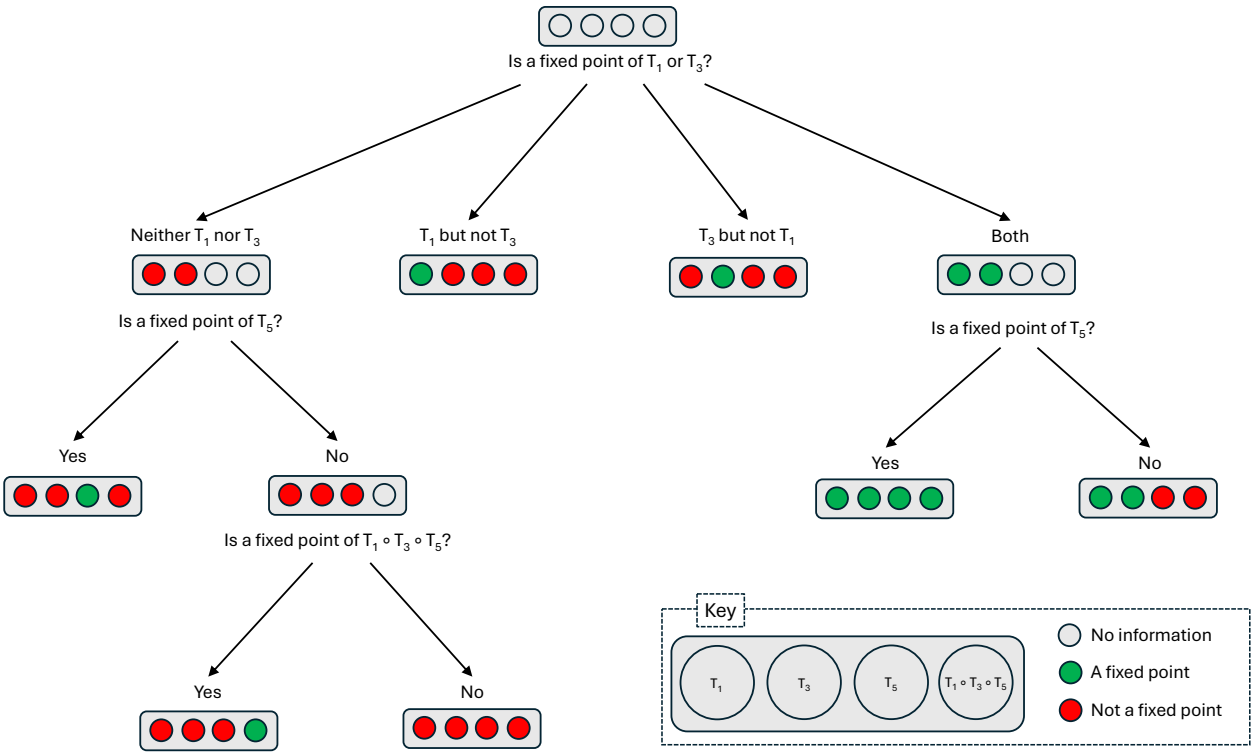


Figure 7: Decision tree to check if a state is a fixed-point of a permutation concerning the blocks fo 4 beds.

5.3 Rewards

The reward received for transitioning from state s_1 to state s_2 given action a was taken, $R(s_1, s_2, a)$, consists of four components:

$$\mathrm{R}(s_1, s_2, a) = -(C(s_1) + P(s_1, \theta))t_{s_1,s_2} - \Psi(a) \tag{24}$$

Where

- t_{s_1,s_2} is the time spent in state s_1 before transitioning to state s_2;
- $C(s_1)$ is the instantaneous staffing cost, corresponding to the number of staff required to care for the patients per time unit. This can be calculated from the ward-state component of s_1, given by Equation 25.
- $P(s_1, \theta)$ is the instantaneous penalty, measured in staff per time unit, corresponding to a Stage 3-I patient not being in an isolation unit. This can be calculated from the ward-state component of s_1, given by Equation 26, where θ is some penalty parameter.
- $\Psi(a)$ is the cost of taking action a, corresponding to either an intra-ward move for patients of type p (cost σ_p), an inter-ward move for patients of type p (cost ϕ_p), or moving a Stage 2 patient to surge (cost ξ). This is given by Equation 27.

$$C = G(M) + \sum_{i=1}^{2} \sum_{j=0}^{16} M_{i,j} \tag{25}$$

$$P(s_1) = \theta \sum_{j=0}^{15} M_{2,j} \tag{26}$$

$$\Psi(a) = \begin{cases} 0 \text{ if } b = d \\ \sigma_p \text{ if } b \text{ in the same ward as } d \\ \phi_p \text{ if } b \text{ in a different ward to } d \\ \xi \text{ if } d = 16 \end{cases} \tag{27}$$

where $s_1 = (M, p)$, $a = (b, d)$, and $G(M)$ represents a function that finds the minimum number of nurses required to cover the Stage 2 patients, when pairs of adjacent patients can be seen by the same nurse, as shown in Figure 4.

5.4 Simulation Model

The simulation model is parametrised as described in Section 3. Additionally we parametrise the isolation penalty with $\theta = 8$.

6 Reinforcement Learning Framework

Reinforcement learning is a model-free unsupervised learning that allows a central agent to learn optimal policies as it explores the state space during a simulation run. One standard algorithm is Q-learning, with overviews given in [10] and [11]. The idea is that each MDP state and action pair, (s, a) with $s \in S$ and $a \in A_s$, has some value $Q(s, a)$, called Q-values, representing the expected long term reward for taking action a when in state s, and following an optimal policy from then on. An optimal policy is, for each state, the action that gives the maximum expected reward, given by Equation 28.

$$a^\star = \underset{a \in A_s}{\operatorname{argmax}}\, Q(s, a) \tag{28}$$

The Q-values are found by iteratively updating them, as the simulation visits states, performs actions, and observes some reward, until convergence. The update, implemented once state s' has been reached, after taking action a in state s, and observing a reward $R(s, s', a)$ is given in Equation 29.

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha \left(R(s, s', a) + \gamma \max_{a' \in A'_s} Q(s', a') \right) \tag{29}$$

Here $\alpha \in (0, 1]$ is the learning rate, a parameter that indicates how important new information is, or how quick the agent is to trust new information; and $\gamma \in [0, 1)$ is the discount factor, a parameter that indicates how important future rewards are as opposed to immediate rewards. The ICU ward is stochastic, and so we anticipate a medium to low learning rate α would be required to overcome the variability in the rewards. The aim is for a general state of high reward throughout the ICU's operating times, and so future rewards are just as important as immediate rewards, so we anticipate a high discount factor γ would be beneficial. In particular, as the rewards are dependent on the time between updates (Equation 24), immediate rewards

are far less meaningful than long run average rewards - for example, one long interval with low instantaneous reward may look better than many shorter periods of high rewards, but it is the long run average that matters.

The reinforcement learning framework used is developed in Python. Due to the very high number of possible states-action pairs, the framework utilises high performance libraries such as Numpy [2] and Numba [8], and was optimised for both speed and memory with help from conversations with Google Gemini [5] to learn and steer some of the software development. It is fully unit-tested and verified, and openly available at `https://github.com/geraintpalmer/bed_moves_policy`.

The Q-values are stored in a Numba typed dictionary, an implementation of a hash table, that maps state-action pairs to Q-values, which are generally kept in memory. At the beginning of a run, all state-action pairs are unexplored, and the Q-value dictionary is empty. When state-action pairs are visited for the first time, the state-action pair is added to the dictionary, mapping it to the Q-value calculated using Equation 29, however this may require the Q-value of other unexplored state-action pairs. In these cases the baseline Q-value for unexplored states will have an effect. Assuming Q-values follow a Normal distribution, we use their ζ-th percentile as a default value, where ζ is a hyper-parameter to choose, Q_ζ, using Equation 30, which can be obtained by keeping track of the running mean and variance of the rewards, $\bar{R}$ and s_R^2, using updates such as Welford's update [12], and the ζ-th percentile of the standard Normal distribution, z_ζ.

$$Q_\zeta = \frac{\bar{R} - z_\zeta s_R^2}{1-\gamma} \tag{30}$$

6.1 State Hashing

Due to the vast number of potential state-action pairs, storing full tuples and matrices in the Q-table is very memory intensive. Therefore in order to preserve memory, we define a reversible hash between state-action pairs $(s, a) = ((M, p), a)$ and 64-bit integers (from -2^{63} to $2^{63} - 1$), that is a 19-digit integer.

We do this with the follow form of mixed radix encoding, given by Equation 31, where $\langle A, B\rangle_F = Tr(A^T B)$ is the Frobenius inner product [1], and the matrix W is given in Equation 32.

$$H((M, p), (b, d))) = 100000\langle M, W\rangle_F + 10000p + 100b + d \tag{31}$$

$$W = \begin{pmatrix} 3\times19\times4^{13} & 2\times19\times4^{13} & 1\times19\times4^{13} \\ 3\times19\times4^{12} & 2\times19\times4^{12} & 1\times19\times4^{12} \\ 3\times19\times4^{11} & 2\times19\times4^{11} & 1\times19\times4^{11} \\ 3\times19\times4^{10} & 2\times19\times4^{10} & 1\times19\times4^{10} \\ 3\times19\times4^{9} & 2\times19\times4^{9} & 1\times19\times4^{9} \\ 3\times19\times4^{8} & 2\times19\times4^{8} & 1\times19\times4^{8} \\ 3\times19\times4^{7} & 2\times19\times4^{7} & 1\times19\times4^{7} \\ 3\times19\times4^{6} & 2\times19\times4^{6} & 1\times19\times4^{6} \\ 3\times19\times4^{5} & 2\times19\times4^{5} & 1\times19\times4^{5} \\ 3\times19\times4^{4} & 2\times19\times4^{4} & 1\times19\times4^{4} \\ 3\times19\times4^{3} & 2\times19\times4^{3} & 1\times19\times4^{3} \\ 3\times19\times4^{2} & 2\times19\times4^{2} & 1\times19\times4^{2} \\ 3\times19\times4^{1} & 2\times19\times4^{1} & 1\times19\times4^{1} \\ 3\times19 & 2\times19 & 1\times19 \\ 9 & 3 & 1 \end{pmatrix}^T \tag{32}$$

The first 14 columns use a counting scheme in base 4, as there are only 4 configurations for each of these columns. Now consider the 15th column, the entries are integers in $\{0, 1, 2\}$, and so the elements of this

column can be counted in base 3. Noting however, due to the constraint that the sum of this column cannot exceed 2, that the maximum value of this column would be 18, corresponding to $(2,0,0)^T$, then we can use base 18 to count this entire column. Together then, M can be represented by by numbers between 0 and 5100273663, that is by at most an 10 digit number. Now using positional concatenation (base 10) we can including the arriving patient type p and action $a = (b, d)$ gives us a 15 digit number, well within the confines of 64-bit integers.

As an example $H((M^\star, 2, (11, 4)) = 507634271021104$, where $M^\star$ is the ward state given in Equation 1.

6.1.1 Choosing Representative States and Actions

In Sections 5.1.1 and 5.2.1 we introduced the idea of representative ward states and representative action, that can stand in place for all ward states and all actions in the same equivalence class. During the simulation run, whenever the algorithm encounters a state, the algorithm will update the Q-value for its equivalent representative state, ensuring faster learning and a smaller exploration space.

The representative states for a particular equivalence class are chosen as those states with the smallest hash. Similarly for actions.

Practically, this means that whenever the simulation reaches state M, then the hash for all states in $[M]$ are found, and the minimum is chosen. Similarly for actions.

6.2 Training Framework

Exploration versus exploitation is an important concept in reinforcement learning, especially in such vast state-spaces. Exploration is important to be able to discover new state-action pairs, however spending too much time exploring can be detrimental, as it wastes time and resources on discovering states that would be seldom visited when following the optimal policy. Exploitation ensures that relevant states are revisited many times, helping to gain enough information on these state-action pairs that their Q-values are updated enough times for these to be a good approximation of the expected future rewards.

Therefore throughout the simulation's training run we choose an action selection scheme that can gradually shift from exploration to exploitation throughout the run. Common schemes include ϵ-greedy and softmax ([10]). The ϵ-greedy scheme exploits with probability ϵ, that is chooses the action with the best Q-value found so far; and explores with probability $1 - \epsilon$; that is chooses uniformly between all the actions. A disadvantage of this is that it enforces a hard cutoff between best and second best, or third best actions - and in such a large state space with stochastic rewards, this can mean that false near-misses are harshly discarded. Softmax addresses this, by choosing actions proportional to their exponential Q-values or ranks. However a disadvantage of this, especially in large action-spaces, is that unexplored actions or those with spurious bad Q-values are pushed to having extremely small selection probabilities.

We address both these issues by implementing a bespoke mixed scheme with exploitation parameter $\epsilon \in [0, 1]$. It explores with probability $1-\epsilon$ according to the ϵ-greedy scheme, and exploits with probability ϵ by choosing between the top three ranked actions, proportional to their exponential Q-values, similar to softmax. For an action a with rank r_a, in an action space of size $|A_s|$, Equation 33 gives the selection probability for this scheme, where the exploitaion weights $(0.4368, 0.3236, 0.2370)$ are generated by applying a rank-based softmax, that is proportional to $e^{-0.3r_a}$ for $r_a \in \{1, 2, 3\}$.

$$\mathbb{P}(a) = \begin{cases} 0.4368\epsilon + \frac{1-\epsilon}{|A_s|} & \text{if } r_a = 1 \\ 0.3236\epsilon + \frac{1-\epsilon}{|A_s|} & \text{if } r_a = 2 \\ 0.2370\epsilon + \frac{1-\epsilon}{|A_s|} & \text{if } r_a = 3 \\ \frac{1-\epsilon}{|A_s|} & \text{otherwise.} \end{cases} \tag{33}$$

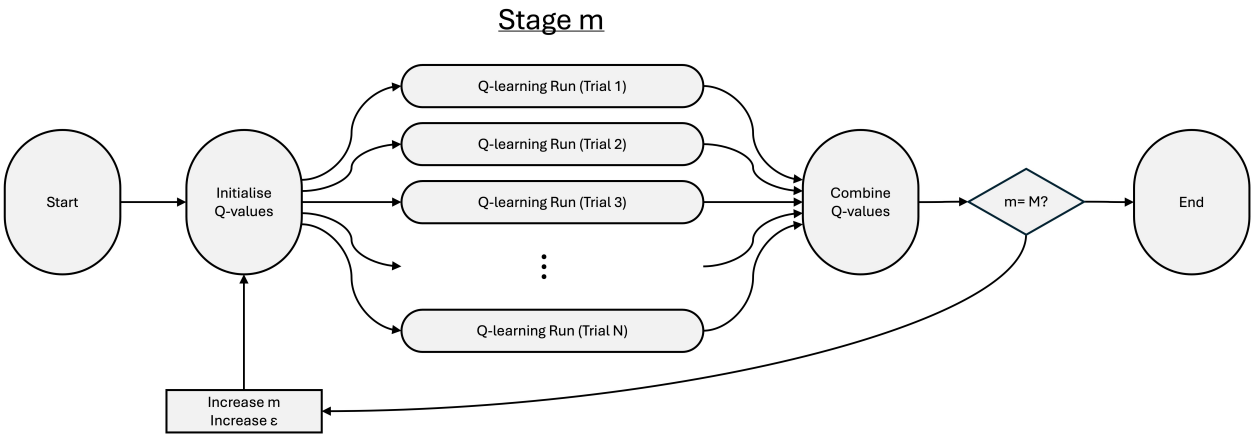


Figure 8: Framework used for training the model.

We choose to begin the training run with a low ϵ and gradually increase it throughout the run - exploring early to find plausible optimal policies, and exploiting later to refine them.

With such a vast state space, and high stochasticity, convergence for all states is practically impossible, and we must settle on very long enough run times to obtain good-enough Q-values to read off optimal policies. These long run times are still computationally expensive and time intensive. We therefore train the agent in a parallel framework, visualised in Figure 8.

We train the model over M stages, utilising N parallel workers. At stage $m = 0$, Q-value tables are initialised as empty, that is no states-action pairs have been discovered yet, and we set $\epsilon = 0$ for full exploration. These Q-value tables are replicated N times, and split over N parallel workers, each running its own training simulation, updating its own table of Q-values for a particular stretch of time. Once these parallel training runs are complete, each worker will have its own distinct table of Q-values: let $Q_n(s, a)$ represent the n^{th} worker's estimate of the Q-value for the (s, a) state-action pair. Workers also keep track of the number of visits to each state-action pair, let $h_n(s, a)$ be the number of times worker n visited that state-action pair. These tables are then combined, weighting each Q-value by the number of visits, that is:

$$Q(s,a) = \frac{\sum_{n=1}^{N} Q_n(s,a) h_n(s,a)}{\sum_{n=1}^{N} h_n(s,a)} \tag{34}$$

In the next stage, we update $\epsilon \leftarrow \epsilon + 1/M$, gradually increasing the agent's rate of exploitation linearly. Then Q-values are initialised with the combined Q-value tables found at the end of the last stage, the h_n are reset to zero, and the training is repeated on parallel workers. This ensures that at the first stage we have full exploration, and by the final stage we have full exploitation, with ϵ increasing linearly over the stages.

After each stage the combined Q-values are saved and so the incumbent policy can be evaluated by running the simulation, without any Q-learning, choosing actions based on that incumbent policy.

There is a danger of over-fitting when training with high exploitation, that is learning 2nd or 3rd best actions which propagate down sequences of state-action choices. Though the mixed action-selection scheme above addresses this during training, evaluation uses a 1.0-greedy scheme, pure exploitation of learned Q-values, which can still highlight the over-fitting. Therefore to ensure that the best policy is found at the end of the training run, the Q-values found at the end of each stage are evaluated, and we take the set of Q-values that minimise the cost. This is equivalent to an early-stopping scheme with infinite patience [4].

As it is standard for any run of a simulation to begin from an empty state, a larger number of parallel trials at each stage yield the added benefit of obliging the agent to sufficiently explore early, near-empty states; despite visits to these states possibly being rarer in reality. This can be beneficial because the agent needs to learn a route from an empty state to the commonly visited states, especially those visited under an optimal policy, to ensure that any time a simulation is run it quickly reaches useful states. This is vital when restarting each stage, but also for policy evaluation.

6.3 Pruning

As the number of theoretically possible state-action pairs is so large, the memory constraints of a computer makes running the algorithm difficult in practice, as the Q-value table grows with exploration. However, the practically reachable space of state-action pairs, when following an optimal, or even just a reasonable, policy, is much smaller. This is often called the 'on-policy distribution' [10]. Therefore any state-action pairs in the suboptimal trajectory space, that is those not in the on-policy distribution, can generally be ignored, reducing the Q-value table and saving memory.

After the end of each stage, these states are identified as those who have not been visited frequently, if at all, and so can be pruned. We prune all state-action pairs whose number of hits $h_n(s, a)$ is less than some chosen limit $\tilde{h}$. This not only helps save memory, but also cleans the data: the simulation is highly stochastic, and for states with $h_n(s, a) < \tilde{h}$, its Q-value will have been based on $h_n(s, a)$ observations only, meaning that for very small $\tilde{h}$ this is just stochastic noise and so cannot meaningfully estimate the expected rewards.

Note that due to the way in which Q-values are passed from stage to stage, it is possible to reach the end of a stage and for a particular state-action pair to be present in the table but have been visited zero times: for example if a pair was visited in stage $n-1$, it was passed to stage n and its number of hits reset to zero, but then not visited at all in stage n. This would be a good candidate to prune and so not pass to stage $n+1$.

7 Experiments

7.1 Parameter Tuning

In order to choose the learning rate α and the discount factor γ we run an hierarchical grid search. Keeping all other parameters equal, in round 1 we first search $(\alpha, \gamma) \in (0.1, 0.2, 0.3, 0.4) \times (0.8, 0.9, 0.95, 0.99)$, and find the optimal combination to be $\alpha = 0.1$ and $\gamma = 0.9$. Then in round 2 we search $(\alpha, \gamma) \in (0.04, 0.08, 0.12, 0.16) \times (0.85, 0.89, 0.8, 0.92, 0.94)$, and refine our optimal combination to $\alpha = 0.16$ and $\gamma = 0.94$, which are the values used throughout the remainder of the paper. The results of this parameter sweep are given in Figure 9.

7.2 Training a Base Policy

The model is trained across 21 stages, each consisting of 60 trials, each training over 400,000 time units, with a default Q-value using$\zeta = 0.5$. For a particular run, each stage' incumbent policy is evaluated by running the simulation for 150 trials, each for a 20,000 time units, and a warm-up time of 7,500 time units, and recording the overall cost running the ICU for the remaining 12,500 time units when exploiting that incumbent policy. An initial 'random' policy is evaluated for comparison. Figure 10 shows these results.

This plot shows an interesting shape wave-like curve which can be described in four stages:

- Random - Stage 1: after the first stage the policy improves, discovering some quick 'low hanging fruit' gains.
- Stage 1 - Stage 4: the policy worsens with training, which may seem unintuitive, but can however be explained by the the choice of default value of unexplored states, using $\zeta = 0.5$: not only is exploration of unexplored state-actions discouraged (only assuming the running median reward), but long-term sequences of actions are penalised if they contain any unexplored state-actions. Hence in

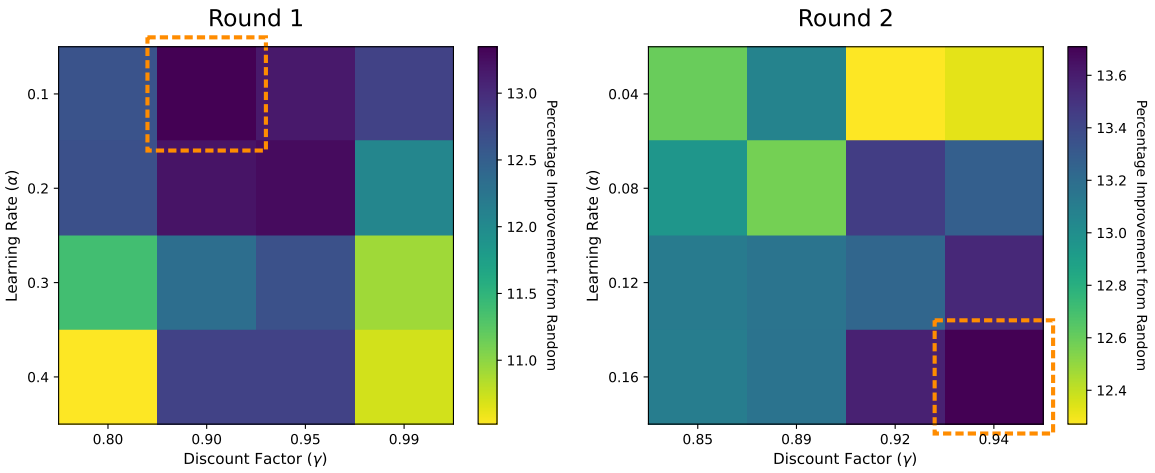


Figure 9: Results of the hierarchical grid search for α and γ.

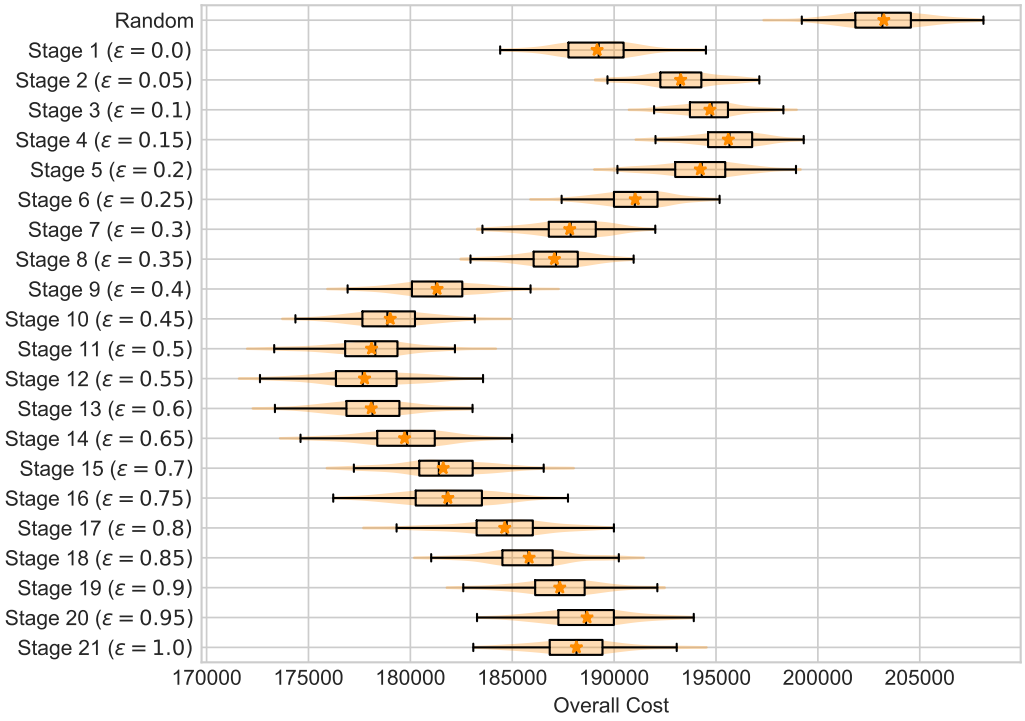


Figure 10: Evaluation of the incumbent policies across the stages.

these partially explored and partially trained regions, fully explored sequences look more optimal than partially explored sequences, even if in reality they are less optimal.

- Stage 4 - Stage 12: Here enough of the state-action space is explored sufficiently that further training improves the policy.
- Stage 12 onward: Here the policy gradually worsens with training again, due to over-fitting when training with such high exploitation, as expected and discussed in Section 6.2. Therefore the early-stopping scheme would choose the incumbent policy at Stage 12 as the optimal.

Overall, this optimal policy produces a median overall cost of 177,663.085, improving on a random (median overall cost of 203,164.96), by 12.6%.

7.3 Effect of Default Q-value

The wave-like training curve seen in Figure 10 can be partially explained by the choice of ζ. Recall that ζ represents the quantile of the rewards found so far in the training run used as the default reward for unexplored states, and therefore the default Q-value associated with those states is given in Equation 30. In Figure 10, $\zeta = 0.5$ was used, that is we take the median reward seed so far as the default. Other values may exhibit different training behaviour.

Figure 11 shows the training curves of three runs using $\zeta \in (0.15, 0.5, 0.85)$, representing pessimistic (assuming unexplored states have worse reward than 85% of what has been observed so far), neutral, and optimistic (assuming unexplored states have better reward than 85% of what has been observed so far). We see with the pessimistic and neutral values that the policies worsen between stages 1 and 4 as exploration is discouraged, pulling the agent into poor local choices. This is avoided completely with the optimistic choice where exploration is encouraged; however it fails to reach the peak performance of the others as over-fitting begins early, that is sequences of best actions do not propagate as they are interrupted by attractive unexplored states. Future work might work to dynamically vary the value of ζ as the training progresses, to capture the best of both pessimistic and optimistic defaults where needed. Note also that although we call $\zeta = 0.5$ neutral, this depends entirely on the distribution of observed rewards, and Figure 11 suggests this may be more pessimistic than originally thought due to its similar behaviour to the pessimistic choice $\zeta = 0.15$.

7.4 Robustness Tests

In practice due to human error or unexpected disruptions it may not be possible to adhere to a given policy 100% of the time. A robust policy would be able to self-correct when a sub-optimal action is chosen, through its following sequence of optimal actions. For MDPs and reinforcement learning problems a fully converged optimal policy is robust by definition, however when training a policy through simulation-optimisation heuristics such as reinforcement learning, especially in a highly stochastic environment such as this, convergence is not guaranteed, and therefore the robustness of the learned policy isn't guaranteed.

In order to test the robustness of the policy, we evaluate the policy using an ϵ-greedy policy, where ϵ is the probability of exploiting, for different values of ϵ. This is shown in Figure 12. We can see that for very small errors, up to around 2%, there is hardly any degradation in performance, however there is then a linear decline in performance up to a 10% error rate, and larger deterioration thereafter.

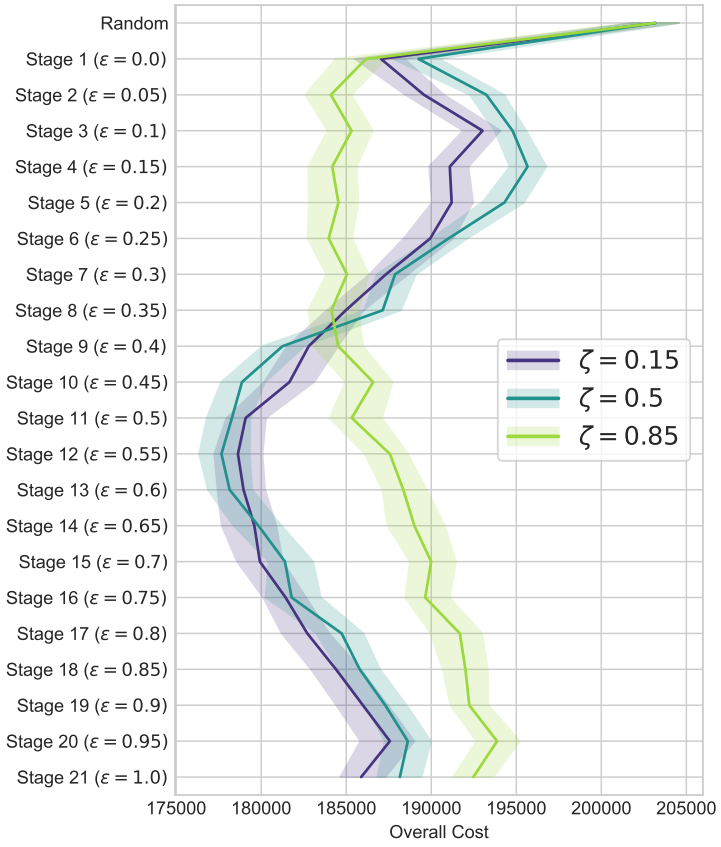


Figure 11: Comparing the training curve using pessimistic, neutral, and optimistic default values for unexplored states. Solid lines denote the median evaluation, while shaded regions show the inter-quartile range.

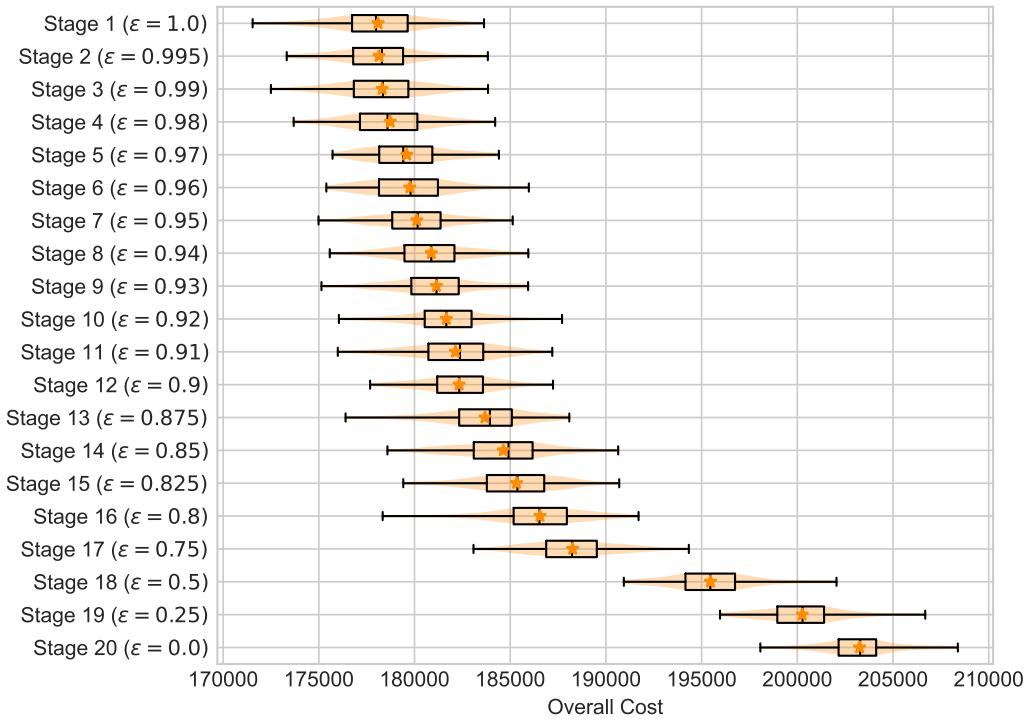


Figure 12: Evaluation of the best policy (incumbent at Stage 12) when using different exploitation probabilities ϵ.

8 Human-Readable Policies

TODO

Idea: Currently the policies that come from the reinforcement learning is a mapping from millions / tens of millions of states to optimal actions. I wonder if we can use some sort of machine learning / clustering algorithm to simplify this into general human-readable ‘guidelines‘ rather than a full detailed policy. This simplified policy would of course be less effective than the full policy, but this is something that can be measured and analysed.

References

[1] Jeff Calder and Peter J Olver. *Linear Algebra, Data Science, and Machine Learning*. Springer, 2025.

[2] Harris CR et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[3] David S. Dummit and Richard M. Foote. *Abstract Algebra*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2004.

[4] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT press Cambridge, 2016.

[5] Google DeepMind. Gemini (3 flash), 2026.

[6] Varun Gupta, Mor Harchol Balter, Karl Sigman, and Ward Whitt. Analysis of join-the-shortest-queue routing for web server farms. *Performance Evaluation*, 64(9-12):1062–1081, 2007.

[7] John FC Kingman. Two similar queues in parallel. *The Annals of Mathematical Statistics*, 32(4):1314–1323, 1961.

[8] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015.

[9] Geraint I Palmer, Vincent A Knight, Paul R Harper, and Asyl L Hawa. Ciw: An open-source discrete event simulation library. *Journal of Simulation*, 13(1):68–82, 2019.

[10] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.

[11] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.

[12] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.